```
public class IndexModel: PageModel
{
    [BindProperty]
    public InputModel Input { get; set; }
    public void OnGet()
    {
    }

    public class InputModel
        {
        public string Category { get; set; }
        public string Username { get; set; }
        public ProductModel Model { get; set; }
    }
}
```

This approach has some organizational benefits that you'll learn more about in section 16.4.

ASP.NET Core automatically populates your binding models for you using properties of the request, such as the request URL, any headers sent in the HTTP request, any data explicitly POSTed in the request body, and so on.

NOTE In this chapter I describe how to bind your models to an incoming request, but I don't show how Razor Pages uses your binding models to help *generate* that request using HTML forms. In chapter 17 you'll learn about Razor syntax, which renders HTML, and in chapter 18 you'll learn about Razor Tag Helpers, which generate form fields based on your binding model.

By default, ASP.NET Core uses three different *binding sources* when creating your binding models in Razor Pages. It looks through each of these in order and takes the first value it finds (if any) that matches the name of the binding model:

- *Form values*—Sent in the body of an HTTP request when a form is sent to the server using a POST
- *Route values*—Obtained from URL segments or through default values after matching a route, as you saw in chapter 14